DSGC: Dynamic Sparse Graph Constrained Autoencoder for Hyperspectral Image Clustering

Feng Liang, Yao Ding, Zhili Zhang, Yaoming Cai, *Member, IEEE*, Jie Feng, *Senior Member, IEEE*, Lianhui Liang, *Member, IEEE*, and Shuli Cheng

***Abstract*—Hyperspectral image (HSI) clustering represents a core unsupervised learning challenge. While deep graph clustering techniques have shown promise for this task, existing approaches typically employ fixed graph network architectures throughout training. This rigidity impedes the adaptation of the initial graph topology to optimize clustering performance. Furthermore, prevailing graph formation strategies frequently generate suboptimal node relationships, producing excessive edges that undermine sparsity. To address these limitations, this article introduces dynamic sparse graph-constrained (DSGC), a DSGC autoencoder framework for HSI clustering. The DSGC methodology begins with local region generation applied to the HSI data. Subsequently, a sparse graph construction phase establishes an initial sparse graph by examining regional connectivity patterns. A dynamic graph autoencoder (GAE) component then extracts high-level semantic representations, incorporating a mechanism for iterative sparse graph refinement. A joint optimization module concurrently enforces structural consistency, enhances graph sparsity, and stabilizes network learning. Latent features derived from this process are clustered using *k*-means. Rigorous experimentation and repeated comparative evaluations confirm that DSGC achieves competitive clustering accuracy, exhibits low computational demands, and demonstrates significant robustness.**

***Index Terms*—Dynamic graph autoencoder (GAE), hyperspectral image (HSI) clustering, joint optimization, sparse graph construction.**

NOMENCLATURE

$\mathcal{G} = (\mathcal{V}, \varepsilon, \boldsymbol{A})$	Graph $\mathcal{G}$ with edge ε and node $\mathcal{V}$.
n	Graph node number.
$\boldsymbol{X} \in \mathbb{R}^{n \times D}$	Node feature matrix.
$\boldsymbol{X}_i$	ith node feature.
$\boldsymbol{A} \in \mathbb{R}^{n \times n}$	Adjacency matrix.
S_i	ith superpixel.
n_i	Number of pixels contained in S_i.
h, w, and b	Height, width, and number of bands of the HSI.
$\boldsymbol{Z}^{(i)}$ and $\boldsymbol{Z}$	ith layer and graph encoder output.
$\boldsymbol{Z} \in \mathbb{R}^{n \times D}$	Clustering-oriented node features.
d_{ij}	Distance between two nodes.
$\sigma(\cdot)$	Activation function.
$p^*(v \mid v_i)$	Conditional probability.
$\boldsymbol{H}^{(l+1)} \in \mathbb{R}^{n \times D'}$	Feature matrices for the $l+1$ layer.
$M_0(\cdot)$	Regularization term.
μ	Distribution freedom degree.
$\boldsymbol{P}$	Auxiliary target distribution.
γ	Coefficient weight.
$\mathcal{L}_c$, $\mathcal{L}_r$	Self-training loss and graph reconstruction loss.

Received 15 April 2026; revised 18 May 2026; accepted 8 June 2026. Date of publication 18 June 2026; date of current version 1 July 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62401632. *(Corresponding author: Yao Ding.)*

Feng Liang, Yao Ding, and Zhili Zhang are with the Key Laboratory of Intelligent Control, PLA Rocket Force University of Engineering, Xi'an 710025, China (e-mail: dingyao.88@outlook.com; 157918018@qq.com).

Yaoming Cai is with the School of Information and Safety Engineering, Zhongnan University of Economics and Law, Wuhan 430073, China (e-mail: caiyaom@zuel.edu.cn).

Jie Feng is with the Key Laboratory of Intelligent Perception and Image Understanding, Ministry of Education of China, Xidian University, Xi'an 710025, China (e-mail: jiefeng0109@163.com).

Lianhui Liang is with the School of Electrical Engineering, Guangxi University, Nanning 530004, China (e-mail: Lianglh0308@gxu.edu.cn).

Shuli Cheng is with the College of Information Science and Engineering and the College of Mathematics and System Science, Xinjiang University, Ürümqi 830046, China (e-mail: cslxju@xju.edu.cn).

Digital Object Identifier 10.1109/TGRS.2026.3705157

I. INTRODUCTION

HYPERSPECTRAL imaging is an advanced technology in the field of remote sensing, which can capture and process electromagnetic spectral information from hundreds of bands, including visible light, near-infrared, and infrared. Due to its ability to detect subtle changes in spectral information, hyperspectral imaging technology has achieved good results in distinguishing different types of land features [1], [2]. Consequently, hyperspectral image (HSI) classification (classifying each pixel in the image into a meaningful category) holds great potential for various applications, e.g., urban planning [3], military detection [4], vegetation monitoring [5], and resource exploration [6]. However, the high dimensionality, strong redundancy, and high computational complexity contained by HSIs also present significant challenges for their analysis, processing, and interpretation [7], [8]. Recently, various mechanisms have been applied for HSI classifiers' design along with artificial intelligence (AI) development, while these classifiers commonly rely on high-quality labeled samples to avoid the underfitting problem resulting from inadequate classifier training [9], [10]. Unfortunately, the manual labeling is commonly costly, labor-intensive, and time-consuming, which severely restricts the applications of HSI. Therefore, the research on unsupervised HSI clustering methods, aiming to learn discriminative features for grouping each pixel into distinct clusters without training labels, has become essential [11].

In prophase, many traditional clustering methods, e.g., *k*-means clustering [12], independent component analysis (ICAs), possibilistic C-means (PCMs) [13], fuzzy C-means [14], spectral clustering (SC) [15], and density peak clustering

1558-0644 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

[16], initially introduced for natural images, have emerged for HSI clustering. Although these methods have shown some effectiveness in HSI clustering, they only focus on spectral features and overlook the spatial structure interconnections of land cover, leading to relatively low clustering accuracy and restricting their widespread applicability [17]. Subsequently, some methods combining spatial spectral information for feature extraction were developed to achieve higher clustering accuracy [18]. For example, in [19], a sparse subspace clustering (SSC) method was developed to enhance clustering efficacy by identifying the representation affinity matrix. Afterward, Zhai et al. [20] introduced an ℓ_2-norm regularized SSC model to further compare spatial information effectively, in which the adjacent information is incorporated into the coefficient matrix. In [21], a scalable graph-based clustering (SGCNR) was proposed to reduce the computational complexity by using nonnegative relaxation.

The aforementioned traditional HSI clustering methods optimize the objective function to obtain the optimal solution, which is simple and has low computational complexity [22], [23], whereas these methods follow the linear assumption and contain shortcomings in extracting deep features, posing significant challenges when facing HSIs with complex nonlinear spectral–spatial features [24], [25]. Subsequently, deep clustering methods were developed to alleviate these shortcomings by extracting robust and profound features. We often can divide most of the deep clustering methods into three modules, i.e., deep feature extractor, representative feature expressor, and self-training optimizer. For example, Zhao et al. [26] proposed deep SC models named DSCRLE, where the regularized linear model was embedded into the network to extract structural features. Cai et al. [27] introduced superpixel-level SC methods named NCSC, in which a contrastive learning and a pooling autoencoder mechanism were adopted to present the latent subspace features. Zhang et al. [28] proposed the elastic graph fusion subspace clustering (EGFSC) methodology. This approach was designed to effectively integrate spectral, spatial, and structural data, facilitating clustering of large HSIs. Nevertheless, in HSI, pixels belonging to the same land cover are widely distributed in spatial dimensions, and land features are interrelated in spatial distribution. Therefore, how to extract latent correlation effectively from spatial structural information is crucial for improving clustering accuracy [29]. Owing to the powerful ability to effectively capture latent spatial relationships between nodes, the graph-based method possesses inherent advantages in structural information modeling, and it has also become the most promising method for feature extraction, processing, and presentation of non-Euclidean data [29]. Therefore, various graph-based clustering works are designed for HSI by introducing structural information through graph embedding. For example, Wang et al. [30] adopted an anchor graph clustering to extract spatial–spectral information with multiscale filtering and a similarity metric. Zhang et al. [31] developed a superpixel-level dual graph autoencoder (GAE) to excavate discriminative information. Ding et al. [32] designed a locality-preserving graph for large-scale HSI, where an autoencoder graph attention network was adopted to extract the latent structure information with a superpixel graph. Peng et al. [33] designed a structural graph to learn the structure features between pixels by using multi-GAE.

While advancements in graph-based clustering techniques for HSI are notable, existing graph-based HSI clustering methods suffer from two critical limitations that hinder their adaptability and performance. First, they predominantly rely on a static graph topology (e.g., k-NN or ϵ-ball graphs) constructed upfront using handcrafted similarity metrics (e.g., Euclidean or spectral distance). Once constructed, this graph remains frozen during the entire training process, preventing the model from correcting initially incorrect or noisy edge connections based on learned representations. Second, most approaches, including those using adaptive graph learning, typically optimize a dense or fixed-sparsity graph (e.g., via ℓ_1 or ℓ_2 regularization) without explicitly enforcing controllable, task-driven sparsity. This leads to redundant edges that introduce computational overhead and obscure discriminative cluster boundaries [34], [35].

To overcome these limitations, this article introduces a dynamic sparse graph-constrained (DSGC) autoencoder framework for HSI clustering. The methodology initiates with a sparse topology construction module designed to derive superpixel-level node attributes, emphasizing interregional dependencies through weighted adjacency matrices. Complementing this, an adaptive GAE framework is proposed to iteratively distill latent embeddings from evolving graph structures, enabling task-specific refinement of connectivity patterns during training. This dual-component system progressively optimizes edge sparsity through cyclic updates, thereby refining the discovery of discriminative relational patterns. Cluster assignments are subsequently generated using a partitioning algorithm applied to the stabilized latent space. Crucially, DSGC integrates a unified optimization mechanism that enforces structural consistency and sparsity constraints while balancing network convergence, ensuring systematic alignment across graph representations and algorithmic efficiency.

The core innovations of DSGC are summarized as follows.

1) A region-aware sparse graph construction strategy is introduced to generate weighted adjacency matrices by evaluating regional connectivity patterns, yielding a compact graph with only k most informative edges per node. This sparsity suppresses noise-induced connections and reduces computational overhead while preserving semantically significant relationships.

2) A trainable autoencoder architecture dynamically adjusts edge weights during optimization cycles, departing from rigid graph configurations. This update mechanism, guided by Kullback–Leibler (KL) divergence metrics, enables responsive adaptation to shifting feature correlations in high-dimensional data.

3) A self-supervised learning strategy leverages auxiliary distributions to stabilize training, while cluster assignments are derived through iterative partitioning of latent embeddings. This end-to-end design ensures synergy between sparse graph refinement and discriminative feature learning.

The subsequent sections of this article are arranged in the following manner. Pertinent notations and definitions

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

are presented in Section II. Section III furnishes a detailed examination of the proposed DSGC framework's key elements. Experimental validation of the approach's effectiveness is conducted in Section IV. Concluding remarks and prospects for future work are discussed in Section V.

II. Notations and Definitions

A. Graph Convolution and Autoencoder

1) Graph Definition: Let $\mathcal{G} = (\mathcal{V}, \varepsilon, \boldsymbol{A})$ be an undirected graph with n nodes, C classes, and edge set ε, where $\mathcal{V} = \{v_1, v_2, \ldots, v_n\}$, and $\boldsymbol{X} \in \mathbb{R}^{n\times D}$ and $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ are the feature matrix and adjacency matrix of the constructed graph, respectively. The main notations adopted in this article and their descriptions are demonstrated in the Nomenclature.

2) Graph Convolution: Graph convolutional networks (GCNs) can represent data as undirected graphs, thereby establishing relationships among different land-cover types. By establishing spatial dependencies for these land covers, GCN further enhances the model's classification performance [36]. For a graph $\mathcal{G} = (\mathcal{V}, \varepsilon, \boldsymbol{A})$, we can express the Laplace matrix $\boldsymbol{L}$ as follows:

$$\boldsymbol{L} = \boldsymbol{D} - \boldsymbol{A}. \tag{1}$$

Then, the normalized Laplace matrix $\boldsymbol{L}_N$ is calculated as follows:

$$\boldsymbol{L}_N = \boldsymbol{I} - \boldsymbol{D}^{-\frac{1}{2}}\boldsymbol{A}\boldsymbol{D}^{-\frac{1}{2}}. \tag{2}$$

GCN introduces a convolutional kernel $g_\theta = diag(\theta)$ to embed nodes, where $\in R^n$. The formula for graph convolution can be expressed as follows:

$$x_G * g_\theta = \boldsymbol{U} g_\theta(\Lambda)\boldsymbol{U}^T x \tag{3}$$

where $x_G \in \boldsymbol{R}^{n\times D}$ is the node representation. Computational simplification is achieved by approximating $g_\theta(\boldsymbol{\Lambda})$ using the Chebyshev polynomial T_K, namely,

$$g_\theta(\Lambda) \approx \sum_{k=0}^{K} \theta_K T_K(\boldsymbol{L}_n). \tag{4}$$

Employing the first-order Chebyshev polynomial streamlines the process, yielding

$$x_G * g_\theta = \sigma\left(\tilde{\boldsymbol{D}}^{-\frac{1}{2}}\tilde{\boldsymbol{A}}\tilde{\boldsymbol{D}}^{-\frac{1}{2}}\right)x. \tag{5}$$

Then, the GCN propagation rule can be expressed as follows:

$$\boldsymbol{H}^{(l+1)} = \sigma\left(\tilde{\boldsymbol{D}}^{-\frac{1}{2}}\tilde{\boldsymbol{A}}\tilde{\boldsymbol{D}}^{-\frac{1}{2}}\boldsymbol{H}^{(l)}\boldsymbol{W}^{(l)}\right) \tag{6}$$

where $\boldsymbol{H}^{(l+1)} \in \mathbb{R}^{n\times D'}$ and $\boldsymbol{H}^{(l)} \in \mathbb{R}^{n\times D'}$ denote the feature matrices for layers $l+1$ and l correspondingly. The symbol $\sigma(\cdot)$ signifies the activation function, while $\boldsymbol{W}$ is the weight matrix.

3) GAE and Clustering: GAEs are unsupervised neural architectures that learn low-dimensional node embeddings by reconstructing graph structural information. Built upon GCNs, GAEs employ an encoder to generate node embeddings through neighborhood aggregation and a decoder to reconstruct adjacency information from the embeddings [37]. The encoder leverages spectral graph convolutions to propagate and transform node features, while the decoder measures pairwise similarity (e.g., via inner products) to predict edge existence. Training minimizes the reconstruction error between the original and decoded adjacency matrices, often optimized through negative sampling for scalability. This enables GAEs to capture topological properties and node attributes for tasks, such as link prediction.

Graph clustering partitions the vertex set $\mathcal{V}$ into distinct classes C, allocating each node $\boldsymbol{v}_i$ to a cluster according to its unique attributes. This process is defined by dual optimization objectives

$$\begin{cases} min\sum_{i=1}^{m}\sum_{j=1}^{m} d(i,j) = min\sum_{i=1}^{m}\sum_{j=1}^{m}\left\|f(\boldsymbol{x}_i) - f(\boldsymbol{x}_j)\right\|_2^2 \\ max\sum_{\alpha=1}^{c}\sum_{\beta=1}^{c} d(\alpha,\beta) = max\sum_{\alpha=1}^{c}\sum_{\beta=1}^{c}\left\|y(\boldsymbol{c}_\alpha) - y(\boldsymbol{c}_\beta)\right\|_2^2 \end{cases} \tag{7}$$

where $f(\cdot)$ generates optimal spatial–spectral embeddings for nodes, while $y(\cdot)$ computes cluster centroids. The variable m indicates the vertex quantity within individual clusters.

B. Graph Construction

Graph networks model interrelationships between nodes through adjacency matrices $\boldsymbol{A}$ that encode topological connectivity. The predominant construction employs a binary adjacency matrix

$$\boldsymbol{A}_{ij} = \begin{cases} 1, & \text{if edge}(i,j) \in \varepsilon \\ 0, & \text{otherwise.} \end{cases} \tag{8}$$

This method is simple and convenient, but the adjacency matrix is not sparse and does not objectively display the degree of closeness between different nodes. More importantly, the adjacency matrix constructed in this way is fixed, which means that the adjacency matrix will not dynamically adjust during the network training process.

III. Proposed Methodology

This section initially outlines the overarching architecture of DSGC, providing a systematic overview of its design principles. Subsequently, its core components are methodically analyzed. Finally, the computational complexity is analyzed.

A. Overarching Architecture

The DSGC framework integrates four core components: local region generation, sparse graph construction, dynamic GAE, and joint optimization mechanisms; the architecture of DSGC, illustrated in Fig. 1, operates as follows.

1) *Local Region Generation:* This module employs a segmentation technique to convert HSIs from pixel-level data into cohesive spatial–spectral units. By leveraging structural homogeneity, it mitigates noise from isolated anomalous pixels and reduces the computational nodes required for subsequent processing.

2) *Sparse Graph Construction:* Diverging from conventional superpixel-based approaches, this phase establishes a weighted graph by analyzing connectivity patterns among regions. The

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

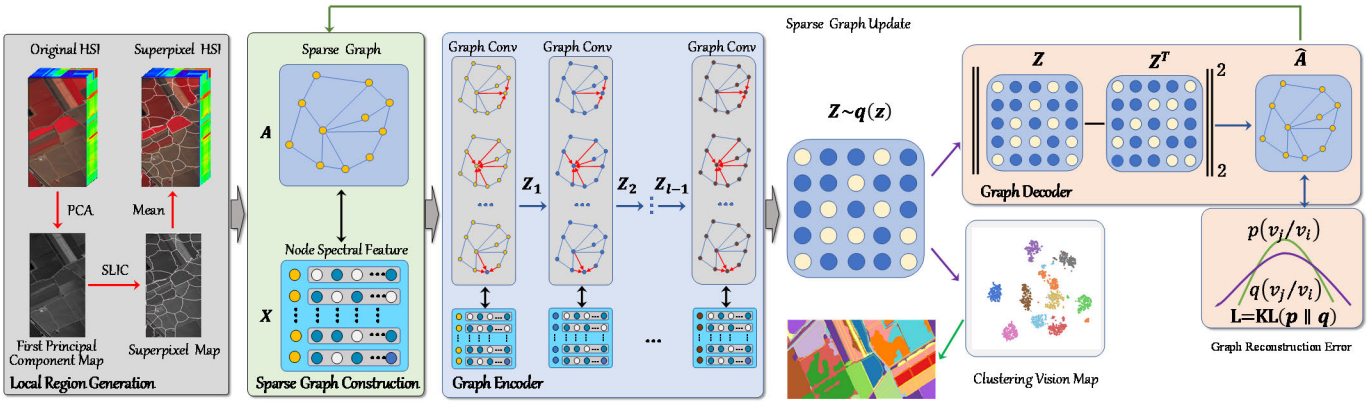


Fig. 1. Overview of DSGC for HSI. The DSGC framework is organized into four key components: local region generation, sparse graph construction, dynamic GAE, and joint optimization mechanisms. Initially, the local region generation phase converts image pixels into spatially coherent regions, preserving local structural integrity. Concurrently, sparse graph construction builds an initial sparse graph structure to represent interregion relationships. A dynamic GAE serves to uncover latent feature representations from the sparse graph, supporting HSI clustering tasks. During training iterations, this component incrementally refines graph sparsity by updating edge connections, thereby enhancing sparse pattern discovery. Finally, the joint optimization module ensures structural uniformity and promotes graph sparsity while harmonizing network training. This integrated approach maintains consistency across graph representations and systematically optimizes the entire architecture for improved clustering performance.

emphasis lies on identifying latent relationships to form a sparse yet meaningful topological representation.

3) *Dynamic GAE:* A GAE is utilized to capture high-level semantic features for clustering. Unlike static models, edge connections are iteratively refined during training via a sparse graph update mechanism, enhancing adaptability to evolving data relationships with an adaptive adjacency matrix.

4) *Joint Network Optimization:* A composite loss function ensures structural coherence and graph sparsity during optimization. Post-training, latent features derived from the encoder are clustered using k-means.

Within DSGC, these components form an end-to-end trainable network, where bidirectional interactions among modules enhance feature representation and clustering efficacy. Each element collaboratively refines the system's ability to model complex hyperspectral data structures.

B. Local Region Generation

The algorithmic complexity in graph-based computations is commonly characterized by $O(N^2)$, indicating that the number of vertices substantially influences the algorithm's processing demands. Within HSI, pixels sharing identical classifications predominantly cluster within statistically uniform areas. Contemporary research underscores the efficacy of superpixel segmentation in elucidating the integrative spectral–spatial attributes of HSIs, facilitating enhanced localization of data homogeneity for advanced analytical workflows. To operationalize this, dimensionality reduction is initially performed on original HSI data via principal components analysis (PCA) [38], producing condensed yet representative imagery for downstream tasks. Subsequently, the simple linear iterative clustering (SLIC) algorithm [39] dynamically partitions these reduced-dimension outputs into irregularly shaped regions, accommodating spatial heterogeneity and textural intricacy. Concluding the workflow, spectral characteristics of individual superpixels are derived through averaging, ensuring consistent feature representation, namely,

$$\begin{aligned} \boldsymbol{X} &= [\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots \boldsymbol{X}_S]^T \\ &= \left[\frac{1}{\tau_1} \sum_{k=1}^{n_1} \boldsymbol{x}_k^1, \frac{1}{\tau_2} \sum_{k=1}^{n_2} \boldsymbol{x}_k^2, \ldots, \frac{1}{\tau_K} \sum_{k=1}^{n_S} \boldsymbol{x}_k^S \right]^T \end{aligned} \tag{9}$$

where $\boldsymbol{X}_i$ and τ_i denote the feature vector of ith superpixel and the number of pixels contained in the superpixel and $\boldsymbol{x}_k^i$ is the spectral feature vector of the pixel.

C. Sparse Graph Construction

Clustering approaches based on graph theory have proven successful in HSI analysis [40]. Nevertheless, many existing techniques predominantly rely on (8) or similar formulations to define the adjacency matrix $\boldsymbol{A} \in \mathbb{R}^{N \times N}$, limiting adaptability post-construction and compromising performance across diverse datasets. To address this, our framework characterizes the fundamental connectivity distribution of a node v_i through a conditional probability $p^*(v \mid v_i)$, normalized such that $\sum_{j=1}^{n} p^*(v_j \mid v_i) = 1$. Under this paradigm, edges between nodes can be interpreted as instances sampled from $p^*(v \mid v_i)$, a foundational premise of generative network embedding.

In practical clustering contexts, connections between nodes are often sparse or absent, necessitating the design of weighted graphs. Here, $p^*(v_j \mid v_i) \geq 0$ provides a probabilistic basis for edge weights. Crucially, the asymmetry $p^*(v_j \mid v_i) \neq p^*(v_i \mid v_j)$ implies that the graph inherently exhibits directionality. Thus, establishing a weighted graph structure essentially involves estimating the intrinsic connectivity distribution. A key theoretical assumption enables approximation of this distribution, facilitating robust clustering outcomes.

Let us denote the distance between two nodes as d_{ij}

$$d_{ij} = d\left(v_i, v_j\right) = \| f\left(\boldsymbol{X}_i\right) - f\left(\boldsymbol{X}_j\right) \|_2^2 \tag{10}$$

where $f(\cdot)$ is a mapping function. The key idea is to view the graph connectivity from a probabilistic perspective: for each node v_i, we aim to define a distribution $p(v_j \mid v_i)$ that

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

represents the probability of the node v_j being connected to v_i. Ideally, nodes that are close in the feature space should have a high connection probability, while nodes far apart should have a near-zero probability. However, directly setting these probabilities based on pairwise distances leads to a dense graph. Therefore, we treat the true connectivity distribution $p^*(v_j \mid v_i)$ as an *unknown* to be estimated, under the constraint that the graph should be sparse and cluster-friendly. If node v_i is close to node v_j, then we expect that

$$\min_{f(\cdot)} \sum_{i=1}^{n} \mathbb{E}_{p(\cdot|v_i)} d\left(v_i, v_j\right) = \sum_{i=1}^{n} \sum_{j=1}^{n} p^*\left(v_j \mid v_i\right) \cdot d_{ij}. \tag{11}$$

To address this challenge, an alternative approach is employed. Direct resolution remains impractical due to the trivial nature of its component problem concerning $p^*(\cdot \mid v_i)$, where $p^*(v_j \mid v_i) = 0$ for $j \neq i$ and $p^*(v_i \mid v_i) = 1$. A widely adopted strategy involves regularized loss minimization, with the objective function formulated as follows:

$$\min_{f(\cdot)} \sum_{i=1}^{n} \mathbb{E}_{p(\cdot|v_i)} d_{ij} + M_0\left(p\left(\cdot \mid v_i\right)\right) \tag{12}$$

where $M_0(\cdot)$ denotes a regularization term. Let $p_*(\cdot \mid v_i)$ represent a prior distribution, and $M_0(\cdot)$ can be decomposed as follows:

$$M_0\left(p\left(\cdot \mid v_i\right)\right) = \gamma M\left(p^*\left(\cdot \mid v_i\right), p_*\left(\cdot \mid v_i\right)\right) \tag{13}$$

where $\gamma > 0$ is a hyperparameter controlling the strength of the regularization term. In the absence of prior knowledge, the prior is typically selected as a uniform distribution $\delta(v_j \mid v_i)$. Here, $M(\cdot, \cdot)$ quantifies the discrepancy between two discrete distributions. While the KL divergence is frequently employed, we propose a simpler metric $M(p^*(\cdot \mid v_i), \delta(\cdot \mid v_i))$, which yields advantageous properties

$$\begin{aligned} M\left(p^*\left(\cdot \mid v_i\right), \delta\left(\cdot \mid v_i\right)\right) &= \sum_{j=1}^{n} \left[p^*\left(v_j \mid v_i\right) - \delta\left(v_j \mid v_i\right)\right]^2 \\ &= \|\boldsymbol{p}_i^*\|_2^2 + \varphi \end{aligned} \tag{14}$$

where $\boldsymbol{p}_i = [p^*(v_1 \mid v_i), p^*(v_2 \mid v_i), \ldots, p^*(v_n \mid v_i)]$ and φ is a constant independent of $p(\cdot \mid v_i)$. Consequently, the objective simplifies to the following equation:

$$\min_{p(\cdot|v_i)} \sum_{i=1}^{n} \mathbb{E}_{p(\cdot|v_i)} d_{ij} + \rho_i \|\boldsymbol{p}_i\|_2^2 \tag{15}$$

where ρ_i regulates sparsity. If $\rho_i = 1/2\left(\alpha d_{i.}^{(\alpha+1)} - \sum_{v=1}^{\alpha} d_{i.}^{(v)}\right)$, where $d_{i.}^{(v)}$ denotes the vth smallest value, the solution guarantees exactly α nonzero entries (i.e., α-sparsity). This yields a closed-form expression

$$p^*\left(v_j \mid v_i\right) = \frac{\left(d_{i.}^{(\alpha+1)} - d_{ij}\right)_+}{\sum_{v=1}^{k} d_{i.}^{(\alpha+1)} - d_{i.}^{(v)}} \tag{16}$$

where α determines the out-degree (number of nonzero edges) for each node, $(\cdot)_+ = max(\cdot, 0)$. Two motivations underpin this design. First, k-sparsity aligns with graph-based clustering objectives, as probabilities for nodes in distinct clusters should approach zero. Second, for this sparse graph construction step, the model's sole hyperparameter α streamlines optimization compared to alternatives requiring extensive tuning.

$p^*(v_j \mid v_i)$ is expressed as (16), and then, we can define the connection between nodes v_i and v_j of the original graph as $\boldsymbol{A}_{ij}$, namely,

$$\boldsymbol{A}_{ij} = \left(p^*\left(v_i \mid v_j\right) + p^*\left(v_j \mid v_i\right)\right)/2. \tag{17}$$

D. Dynamic GAE

1) Graph Embedding Convolutional Encoder: The input graph's adjacency matrix $\boldsymbol{A}$ is formulated as per (17). Subsequent latent representations are iteratively derived through spectral graph convolutional layers. The output of the lth layer is governed by the operation

$$\boldsymbol{Z}^{(l)} = GCN\left(\boldsymbol{Z}^{(l-1)}; \boldsymbol{A} \mid \boldsymbol{W}^{(l)}\right) \tag{18}$$

where $\boldsymbol{Z}^{(l-1)}$ and $\boldsymbol{Z}^{(l)}$ represent the input and output feature matrices of the lth layer, respectively, and $\boldsymbol{W}^{(l)}$ denotes the trainable parameter matrix optimized during network training. Following the methodology outlined in (18), the initial feature matrix $\boldsymbol{Z}^{(0)}$ is set equivalent to the input node attributes $\boldsymbol{X}$. As derived in (6), the graph convolution operation for each layer is mathematically defined as follows:

$$GCN\left(\boldsymbol{Z}^{(l-1)}; \boldsymbol{A} \mid \boldsymbol{W}^{(l)}\right) = \sigma\left(\tilde{\boldsymbol{D}}^{-\frac{1}{2}} \tilde{\boldsymbol{A}} \tilde{\boldsymbol{D}}^{-\frac{1}{2}} \boldsymbol{Z}^{(l-1)} \boldsymbol{W}^{(l)}\right) \tag{19}$$

where $\sigma(\cdot)$ signifies an activation function and $\tilde{\boldsymbol{A}} = \boldsymbol{A} + \boldsymbol{I}$ (with $\boldsymbol{I}$ as the identity matrix) represents the normalized adjacency matrix incorporating self-connections.

The final latent representation $\boldsymbol{Z}$ produced by stacking m convolutional layers is computed as follows:

$$\boldsymbol{Z} = \sigma_m\left(\hat{A}\sigma_{m-1}\left(\cdots \sigma_1\left(\hat{\boldsymbol{A}}\boldsymbol{X}\boldsymbol{W}_1\right)\cdots\right)\boldsymbol{W}_m\right) \tag{20}$$

where $\hat{\boldsymbol{A}} = \tilde{\boldsymbol{D}}^{-1/2} \tilde{\boldsymbol{A}} \tilde{\boldsymbol{D}}^{-1/2}$ constitutes the symmetrically normalized adjacency matrix, ensuring stable gradient propagation during training.

2) Dynamic Graph Decoder: Unlike existing adaptive graph methods, such as DAEGC [41] and EGAE [42], which either keep the graph topology fixed or learn dense edge weights, our decoder is designed to dynamically adjust both edge weights and the underlying sparse topology in response to the clustering objective. Traditional methods typically employ inner products to reconstruct $\boldsymbol{A}$ to achieve the decoding process, which results in $\boldsymbol{A}$ remaining unchanged throughout the entire training process. In this article, we obtain the adjacency matrix $\boldsymbol{A}$ by calculating the distribution of connections between nodes $p^*(v_j \mid v_i)$. Specifically, the updated adjacency matrix $\boldsymbol{A}^{(t|1)}$ is computed as follows:

$$\boldsymbol{A}^{(t|1)} = (1 - \eta) \cdot \boldsymbol{A}^{(t)} + \eta \cdot \boldsymbol{Q}^{*(t)} \tag{21}$$

where $\boldsymbol{Q}^{*(t)}$ is the matrix of $q^*(v_j \mid v_i)$ at iteration t and $\eta \in [0, 1]$ is a smoothing factor controlling the update rate. This ensures that the graph structure evolves gradually, preserving stability while adapting to learned feature representations.

To extract deep features from graph networks, it is necessary to restore $p^*(v_j \mid v_i)$. Therefore, we define d_{ij} to present the distances of latent representation $\boldsymbol{Z}$ as follows:

$$d_{ij} = \left\|z_i - z_j\right\|_2^2 \tag{22}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

where z_i is the ith column vector of $\boldsymbol{Z}$. Then, a normalization is adopted to reconstruct the connectivity distribution, namely,

$$q^*\left(v_j \mid v_i\right) = \frac{\exp\left(-d_{ij}\right)}{\sum_{j=1}^{n} \exp\left(-d_{ij}\right)}. \tag{23}$$

This procedure can be interpreted as passing $-d_{ij}$ through a SoftMax layer. Notably, smaller d_{ij} values yield larger $q^*(v_j \mid v_i)$, indicating that similarity is determined by the Euclidean distance rather than the inner products commonly employed in GAEs. To quantify the discrepancy between the two distributions, we employ KL divergence, formulating the objective function $\mathcal{L}_r$ as follows:

$$\begin{aligned}\mathcal{L}_r &= \min_{q(\cdot|v_i)} KL(p^*q^*) \\ &= \min_{q(\cdot|v_i)} \sum_{i,j=1}^{n} p\left(v_j \mid v_i\right) \log \frac{1}{q^*\left(v_j \mid v_i\right)}.\end{aligned} \tag{24}$$

By using (24), we can adaptively adjust the adjacency matrix according to different clustering tasks as the network is trained, that is, adjust the connection values between different nodes to change the sparsity of the graph, thereby improving clustering accuracy. The KL divergence loss $\mathcal{L}_r$ is used to guide the reconstruction of the connectivity distribution, but the actual graph update is performed explicitly using the moving average strategy defined above. This avoids trivial solutions and stabilizes training.

E. Joint Optimization Mechanisms

To address unsupervised node clustering in graph representation learning, this work integrates a dynamic GAE with self-supervised training. While the encoder generates latent graph embeddings through dimensionality reduction, conventional approaches neglect cluster-oriented optimization during representation learning. A critical obstacle in unsupervised scenarios arises from the absence of supervisory signals for model training. To bridge this gap, we have implemented a self-training paradigm that utilizes iterative label refinement to guide cluster formation. The clustering objective adopts the KL divergence

$$\mathcal{L}_c = KL(\boldsymbol{P} \parallel \boldsymbol{Q}) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \tag{25}$$

where $\boldsymbol{P}$ denotes the target distribution and $\boldsymbol{Q}$ represents soft assignments derived from node-cluster similarity. Specifically, students' t-distribution serves as the kernel function for probabilistic assignments

$$q_{ij} = \frac{\left(1 + \|z_i - u_j\|^2/\mu\right)^{-\frac{\mu+1}{2}}}{\sum_{j'} \left(1 + \|z_i - u_{j'}\|^2/\mu\right)^{-\frac{\mu+1}{2}}} \tag{26}$$

where μ indicates the distribution freedom degree, with q_{ij} quantifying node i's affiliation to cluster j. These assignments form matrix $\boldsymbol{Q} \in \mathbb{R}^{n \times C}$.

To enhance cluster compactness, an auxiliary target distribution $\boldsymbol{P}$ sharpens assignments through

$$p_{ij} = \frac{q_{ij}^2 / \sum_i q_{ij}}{\sum_{j'} \left(q_{ij'}^2 / \sum_i q_{ij'}\right)}. \tag{27}$$

Minimizing $\mathcal{L}_c$ promotes intracluster cohesion and intercluster separation by densifying $\boldsymbol{Q}$. The composite optimization function combines reconstruction and clustering objectives

$$\mathcal{L} = \underbrace{\mathcal{L}_r}_{Reconstruction} + \underbrace{\lambda \mathcal{L}_c}_{Clustering} \tag{28}$$

where hyperparameter λ balances component weights.

Implementation involves initializing centroids $\{\mu_j\}_{j=1}^{c}$ via k-means on embeddings $\boldsymbol{Z}$, with p_{ij} refreshed periodically through (24)–(26). Node labels derive from maximum assignment probabilities

$$y_i = \arg \max_j q_{ij}. \tag{29}$$

Algorithm 1 DSGC Autoencoder for Hyperspectral Image Clustering

Input: Graph $(\mathcal{V}, \mathcal{E}, \boldsymbol{A})$; Cluster number C; Iterations Number T ; Coefficient weight λ ; Distribution threshold δ.
Output: Clustering results $\boldsymbol{Q}$.
Construct the graph by Eq.(17).
Initialize $\mathbf{Z} = \boldsymbol{X}$, $\alpha = \alpha_0$. Implementation involves initializing centroids $\{\mu_j\}_{j=1}^{c}$ via k -means on embeddings $\mathbf{Z}$.
Initialize clusters centroid μ_j with k-means clustering based on $\mathbf{Z}$.
For t=1 to T do
  Perform graph autoencoder by Eq. (20) and Eq. (23) to obtain the graph hidden presentation $\boldsymbol{Z}$.
  Update adjacency matrix by Eq. (21).
  Calculate sparse graph update objective function $\mathcal{L}_r$ with Eq.(24).
  Calculate clustering loss $\mathcal{L}_c$ with Eq.(25).
  if $L_c \leq \delta$ **break**;
  Calculate The composite optimization function $\mathcal{L}$ and update the sparse graph and weight matrices by Adam gradient descent;
end
Obtain final optimized soft labels distribution $\boldsymbol{Q}$ (final clustering results) with Eq. (29).

Training terminates when label distribution changes between iterations fall below threshold δ or upon reaching epoch limits. Final clusters emerge from stabilized $\boldsymbol{Q}$ values, with full algorithmic details provided in Algorithm 1. This formulation achieves dual objectives: maintaining topological fidelity through reconstruction while inducing discriminative embeddings via self-training cluster refinement. The t-distribution kernel provides robustness to outlier nodes during probabilistic assignments, and the adaptive target distribution progressively clarifies cluster boundaries through training iterations.

F. Computational Complexity Analysis

Consider a graph with n nodes, where d_i denotes the spectral dimension of nodes in the ith layer, $|\varepsilon|$ represents the count of nonzero entries in the adjacency matrix $\boldsymbol{A}, C$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

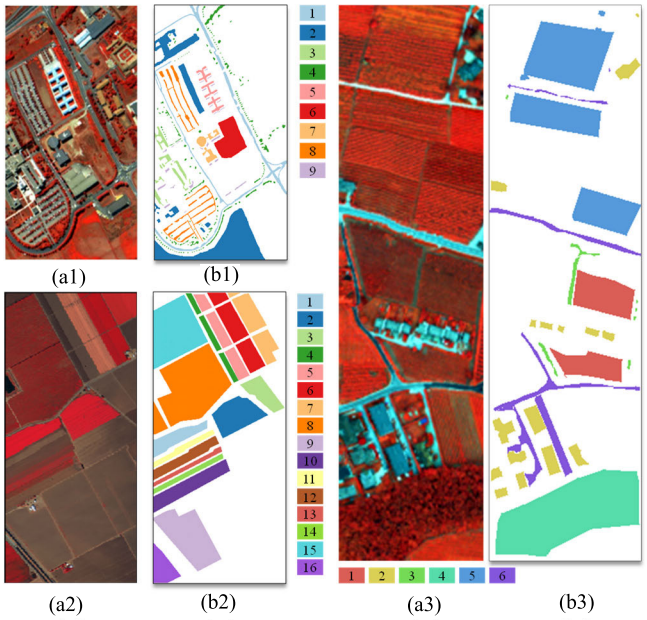


Fig. 2. Details of PU, SA, and Trento. (a1)–(a3) False-color image of three HSI datasets, respectively. (b1)–(b3) Corresponding ground-truth image.

indicates the number of clusters, and l corresponds to the number of layers in a graph-embedded autoencoder. For a multilayer GCN architecture, the computational cost can be quantified as $O\left[(n+|\varepsilon|)\sum_{i=1}^{l} d_i d_i\right]$. In contrast, the primary computational burden for k-means clustering lies in $O(ndlC)$. Assuming T iterative cycles for the model, the overall computational complexity of DSGC is expressed as $O\left[\left[(n+|\varepsilon|)\sum_{i=1}^{l} d_i d_i + ndlC\right]T\right]$.

IV. Experiment and Result Analysis

In this section, extensive experiments and analyses are introduced to assess the clustering performance of the proposed DSGC. Specifically, the three well-known HSI datasets and experimental setups are first introduced (see Sections IV-A and IV-B). Subsequently, the quantitative and visual HSI clustering performances of DSGC are compared with nine state-of-the-art algorithms (see Section IV-C). Moreover, we implement some ablation experiments to evaluate the rationality of the proposed DSGC design (see Section IV-D). Furthermore, the influence of hyperparameters is analyzed (see Section IV-E). In addition, the clustering visualizations are demonstrated (see Section IV-F), and finally, the computational complexity of deep clustering methods investigated is compared and analyzed (see Section IV-G).

A. Datasets

The performance of DSGC was assessed using three benchmark hyperspectral datasets: Pavia University (PU), Salinas (SA), and Trento. Clustering outcomes and statistical measures are employed for this evaluation. Visual depictions, including false-color representations alongside their associated reference data for these datasets, are presented in Fig. 2.

TABLE I
Preset Optimal Parameters in AHSGC

Dataset	n	l	T	L	γ
Salinas	530	4	50	5e-4	0.4
PU	750	4	50	5e-4	0.4
Trento	520	4	50	5e-4	0.4

The Salina Hyperspectral dataset (SA) originates from AVIRIS sensor imagery collected in Salinas Valley, California. Featuring high spatial resolution, the dataset spans a region of 512×217 pixels, initially comprising 224 spectral channels, with 204 retained post-exclusion of noise-affected bands. It encompasses 16 distinct land cover categories, totaling 111 104 pixels, of which 56 975 are unclassified background pixels and 54 129 are labeled for classification.

Acquired via the ROSIS-03 airborne sensor in 2003, the PU dataset represents a hyperspectral subset from Pavia, Italy. Covering a spectral range of 0.43–0.86 μm across 115 bands at 1.3-m spatial resolution, the data exclude 12 bands due to noise, yielding 103 usable channels. With dimensions of 610×340 (207 400 pixels), only 42 776 pixels correspond to nine distinct ground object classes, reflecting a low proportion of annotated data relative to the total.

The Trento dataset, captured by the AISA Eagle sensor in rural southern Trento, Italy, comprises 600×166 pixels and 63 spectral channels within 0.40–0.98 μm. Characterized by 1-m spatial resolution, this dataset provides detailed spectral information for agricultural and environmental analysis. Each sensor configuration underwent radiometric calibration to ensure data fidelity across varying illumination conditions.

B. Experiment Setup

1) Experimental Settings: The benchmarking framework employs several metrics to assess classification and clustering performance, including overall accuracy (OA), kappa coefficient (κ), normalized mutual information (NMI), adjusted Rand index (ARI), and purity. OA quantifies the proportion of correctly classified instances relative to the total dataset. The κ coefficient evaluates interrater agreement, accounting for chance alignment. NMI gauges the statistical dependence between two cluster distributions, while ARI estimates the congruence between algorithmic results and ground-truth labels. As a transparent measure, purity reflects the dominant class proportion within clusters, scaled between 0 and 1, with higher values indicating superior clustering outcomes.

2) Parameter Settings and Implementation Details: To ensure equitable benchmarking, experimental trials are executed on a standardized computational setup utilizing the PyTorch framework. The hardware configuration comprised an Intel Xeon Silver 4210R central processing unit, 128 GB of system memory, and an NVIDIA Titan RTX graphics processing unit with 24 GB of VRAM.

In our DSGC, five main parameters, i.e., graph number n, encoder layer number l, iteration number T, learning rate L, and coefficient weight γ, should be preset. Our proposed DSGC uses the Adam optimizer with a batch size of 64. Their

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

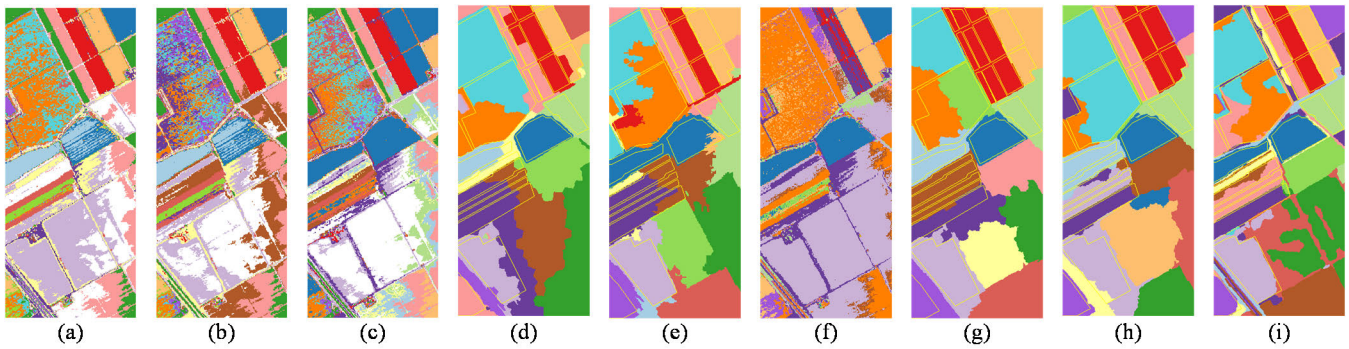


Fig. 3. Clustering maps of the SA dataset. (a) *k*-means. (b) FCM. (c) PCM. (d) DFCN. (e) NCSC. (f) SDCN. (g) EGAE. (h) DAEGC. (i) DSGC.

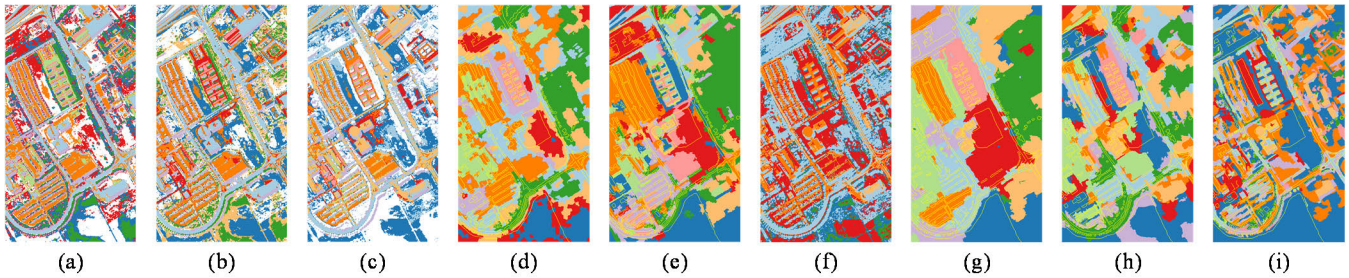


Fig. 4. Clustering maps of the PU dataset. (a) *k*-means. (b) FCM. (c) PCM. (d) DFCN. (e) NCSC. (f) SDCN. (g) EGAE. (h) DAEGC. (i) DSGC.

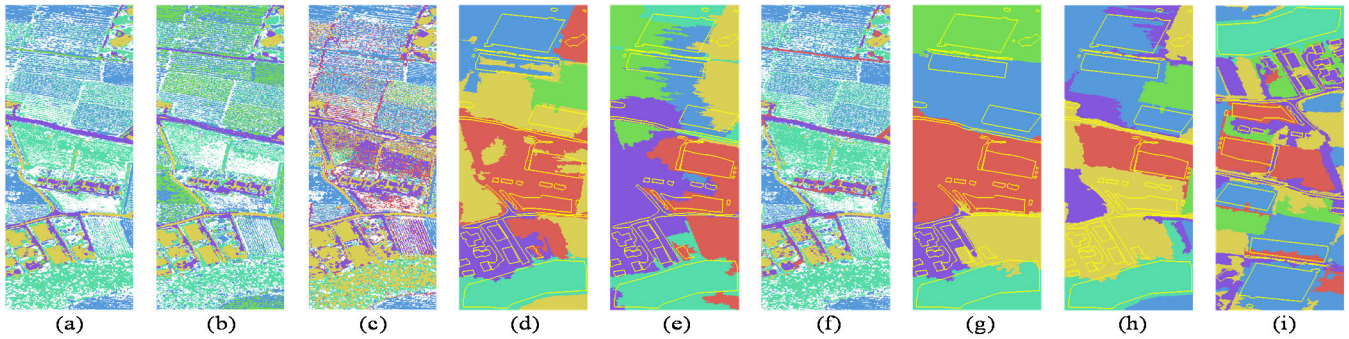


Fig. 5. Clustering maps of the TR dataset. (a) *k*-means. (b) FCM. (c) PCM. (d) DFCN. (e) NCSC. (f) SDCN. (g) EGAE. (h) DAEGC. (i) DSGC.

optimal values are shown in Table I and will be analyzed in Section IV-D.

3) Compare Baseline: To thoroughly evaluate the proposed method's performance, DSGC is compared against eight established benchmarks. These included conventional clustering algorithms (*k*-means [12], fuzzy C-means [14], and PCMs), subspace/graph techniques (NCSC [27] and DAEGC [41]), and deep learning approaches (DFCN [43], SDCN [44], and EGAE [42]). Hyperparameter settings and neural network configurations for all comparative techniques adhered to specifications within their respective source publications.

C. Comparative Experimental Results and Analysis

This section evaluates DSGC against eight alternative clustering techniques, showcasing its effectiveness through quantitative and qualitative assessments. The quantitative assessments are demonstrated in Tables II–IV. For clarity, top and second-best performances are highlighted using bold and underlined formatting. Furthermore, the corresponding quantitative assessments are shown in Figs. 3–5.

1) Clustering Results on the SA Dataset: Table II details comparative method performances on the SA dataset. Analysis reveals DSGC secures the second-highest rank overall, achieving scores of 77.83% (OA), 74.23% (kappa), 82.68% (NMI), 65.83% (ARI), and 80.29% (purity). Methods including DFCN, NCSC, EGAE, and DAEGC also yield reasonably strong outcomes, indicating their robustness against noise and class imbalance. Graph-based approaches (DAEGC, EGAE, and DSGC) perform well, likely due to their proficiency in modeling node interconnections. Conversely, conventional techniques (*k*-means, FCM, and PCM) demonstrate notably inferior results, attributed to their inability to learn complex hyperspectral features, evidenced by OAs of 67.99%, 56.73%, and 55.63%. DSGC's performance is further enhanced by the dynamic GAE, which leverages a sparse graph update strategy to acquire high-level semantic representations for clustering.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

TABLE II

SALINA DATASET CLUSTERING PERFORMANCE. PRIMARY OUTCOMES ARE DENOTED BY BOLD TEXT; UNDERSCORES SIGNIFY SECONDARY PERFORMANCE LEVELS

No.	*k*-means	FCM	PCM	DFCN	NCSC	SDCN	EGAE	DAEGC	DSGC
1	0.9985	0.4179	0.6456	**1.0000**	0.0000	0.0000	**1.0000**	**1.0000**	0.0000
2	0.5698	0.5524	0.8125	0.9990	**1.0000**	**1.0000**	**1.0000**	0.9997	0.9987
3	0.9741	0.8501	0.0000	**1.0000**	0.6822	0.3426	**1.0000**	0.0000	**1.0000**
4	0.9865	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	**1.0000**
5	0.7856	0.5540	0.9628	0.7308	**0.9988**	0.9816	0.0000	0.6502	0.9746
6	0.9952	0.9990	0.9906	0.9986	0.9941	0.9530	0.9886	**1.0000**	0.9866
7	0.4810	0.0000	0.9943	0.9561	**1.0000**	0.0000	0.9983	0.9718	0.9999
8	0.7309	0.7772	0.3875	0.4320	**0.8026**	0.7633	0.5131	0.6683	0.5631
9	**0.9750**	0.6969	0.5829	0.9063	0.9076	0.8643	0.8326	0.9141	0.9615
10	0.6169	0.5507	0.4831	0.7215	**0.9900**	0.5519	0.8962	0.2379	0.9138
11	0.3370	0.0000	0.7462	0.5511	**0.9531**	0.0000	0.0000	0.0000	0.6175
12	0.0273	0.0047	0.8357	0.5051	0.0000	0.2287	**1.0000**	0.8583	0.4312
13	0.3548	0.8134	**0.9582**	0.0656	0.0000	0.0690	0.0000	0.0000	0.0000
14	0.5584	0.6512	0.1390	0.0013	0.0000	0.7632	0.0000	0.0000	**0.9850**
15	0.4994	0.4279	0.2570	**0.9477**	0.8563	0.0980	0.9692	0.8176	0.9451
16	0.0000	0.8321	0.4382	0.2911	**1.0000**	0.0000	**1.0000**	0.9723	0.9562
OA (%)	0.6799	0.5673	0.5563	0.7193	**0.7881**	0.5382	0.7361	0.7066	0.7783
Kappa	0.6572	0.5431	0.5472	0.6900	**0.7602**	0.4793	0.7106	0.6741	0.7423
NMI	0.7356	0.6859	0.6631	0.8288	**0.8514**	0.6589	0.8769	0.7841	0.8268
ARI	0.5466	0.4693	0.4539	0.6278	**0.7068**	0.4831	0.7183	0.6096	0.6583
Purity	0.7132	0.7016	0.6708	0.7324	0.7893	0.5765	0.8357	0.7129	**0.8029**

TABLE III

PU DATASET CLUSTERING PERFORMANCE. PRIMARY OUTCOMES ARE DENOTED BY BOLD TEXT; UNDERSCORES SIGNIFY SECONDARY PERFORMANCE LEVELS

No.	*k*-means	FCM	PCM	DFCN	NCSC	SDCN	EGAE	DAEGC	DSGC
1	**0.8234**	0.5732	0.5538	0.4051	0.4238	0.6234	0.3029	0.3759	0.3759
2	0.3581	0.3265	0.4633	0.4008	0.5933	0.3803	0.5012	0.7935	**0.7948**
3	0.2167	0.0024	0.6027	0.7038	0.6902	0.1687	**0.9900**	0.9609	0.9609
4	0.5943	0.5367	0.1648	0.3547	0.0270	0.4025	0.2744	0.6505	**0.6513**
5	0.6620	0.7124	0.0000	0.6892	0.9735	0.6610	**1.0000**	**1.0000**	**1.0000**
6	0.4255	0.3529	0.3265	0.4940	0.9711	0.3328	**1.0000**	0.3327	0.3327
7	0.0000	**0.9578**	0.0068	0.3486	0.3382	0.3622	0.0000	0.5301	0.5482
8	**0.9608**	0.8730	0.5538	0.3444	0.4027	0.4729	0.6258	0.1645	0.1645
9	**0.9990**	0.0000	0.9989	0.1683	0.0000	0.3702	0.0000	0.0847	0.0847
OA (%)	0.5237	0.4359	0.4376	0.4214	0.5519	0.4207	0.5328	0.6010	**0.6022**
Kappa	**0.5246**	0.4327	0.4261	0.3189	0.4570	0.3185	0.4536	0.4866	0.5047
NMI	0.5529	0.5126	0.4758	0.4355	0.4392	0.4362	0.5372	0.5142	**0.5952**
ARI	0.3188	0.2632	0.2783	0.2691	0.3869	0.2399	0.3780	**0.5049**	0.4314
Purity	**0.7012**	0.6785	0.6487	0.5966	0.6638	0.6400	0.7126	0.6842	0.6770

2) Clustering results on the PU dataset: Land cover in the PU dataset is more fragmented than in SA, presenting greater clustering difficulties. Reflecting this, Table III indicates degraded results across all methods. Despite this, DSGC maintains superior performance. It improves upon the best competitor by 0.12% in OA and 4.23% in NMI. Interestingly, deep and graph-based models show limited advantage over traditional ones here. This stems from the dataset's dispersed yet interrelated land features, complicating class separation and underscoring the necessity for extracting highly discriminative features. Crucially, DAEGC and DSGC achieve notable breakthroughs (OAs of 60.10% and 60.22%, respectively), attributed to their GAE mechanisms effectively capturing latent structural information within the data.

2) Clustering Results on the Trento Dataset: Relative to SA and PU, the Trento dataset exhibits a simpler land cover distribution. Nevertheless, as indicated in Table IV, conventional methods (*k*-means, FCM, and PCM) yield poor outcomes (OAs: 63.01%, 51.32%, and 36.78%) due to inadequate feature extraction. DSGC excels, attaining an OA of 78.34% and surpassing the next best method by 3.45%. This superiority stems from its potent feature extraction and cluster discrimination capabilities. DSGC's sparse graph update mechanism facilitates accurate estimation of node-class relationships, enabling adaptive edge connection refinement that boosts clustering precision. Furthermore, its self-supervised structure and feature loss function facilitate effective self-representation learning for HSI data.

D. Parameters Analysis

In this section, the impacts of different hyperparameters contained in DSGC on clustering performance are investigated. The graph number n, encoder layer number l, iteration number T, learning rate L, the sparsity coefficient α, and the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
TRENTO DATASET CLUSTERING PERFORMANCE. PRIMARY OUTCOMES ARE DENOTED BY BOLD TEXT; UNDERSCORES SIGNIFY SECONDARY PERFORMANCE LEVELS

No.	k-means	FCM	PCM	DFCN	NCSC	SDCN	EGAE	DAEGC	DSGC
1	0.0000	0.0000	0.3673	0.6844	**1.0000**	0.2218	0.6522	**1.0000**	0.9596
2	0.0189	0.0000	0.0000	0.4191	0.0793	**0.7826**	0.6029	0.1151	0.0134
3	0.0000	0.0000	0.0000	0.0063	0.0000	0.0359	**0.0931**	0.0000	0.0146
4	0.7200	0.6826	0.4502	0.9745	**1.0000**	0.6237	0.9540	**1.0000**	0.9984
5	0.6182	0.2007	0.4292	0.7105	0.6437	**0.7452**	0.5196	0.6156	0.7156
6	0.0199	0.0329	0.0006	0.3959	0.8206	0.5930	0.3412	0.8233	**0.9865**
OA (%)	0.6301	0.5132	0.3678	0.7145	0.7489	0.6200	0.6487	0.7469	**0.7834**
Kappa	0.6026	0.4917	0.3232	0.6314	0.6790	0.5132	0.5380	0.6741	**0.7187**
NMI	0.4928	0.4654	0.2898	0.6787	0.7312	0.4860	0.6801	0.6767	**0.7941**
ARI	0.3457	0.2802	0.1303	0.6795	0.7111	0.4362	0.6624	0.6483	**0.7517**
Purity	0.6499	0.6708	0.5146	0.7817	**0.8652**	0.7050	0.8273	0.8382	0.8534

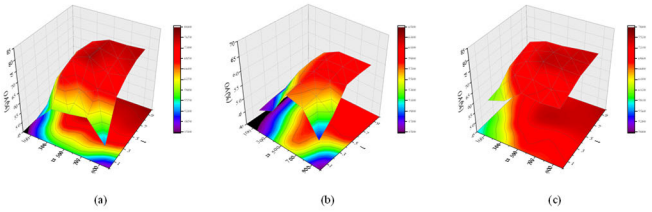


Fig. 6. Sensitivity analysis of DSGC to n and l on (a) SA, (b) PU, and (c) Trento.

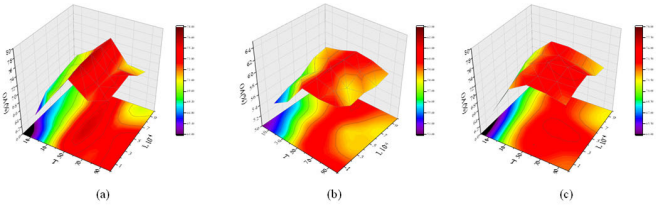


Fig. 7. Sensitivity analysis of DSGC to T and L on (a) SA, (b) PU, and (c) Trento.

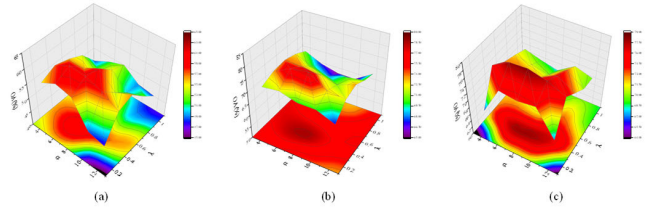


Fig. 8. Sensitivity analysis of DSGC to α and λ on (a) SA, (b) PU, and (c) Trento.

loss weight λ are explored through a grid search strategy. Figs. 6–8 demonstrate the variations in OA on three datasets with different values of $(n, l), (T, L)$, and (α, λ), respectively. From the results, we can observe that four parameters have a significant impact on final clustering accuracy, and the model is more likely to achieve superior performance when the $l \in [3, 5], T \in [40, 60]$, $L \in [3e^{-4}, 5e^{-4}]$, and $\alpha \in [6, 8]$. In addition, based on the experimental results, n's are set to 530, 750, and 520 for SA, PU, and Trento, respectively. When $\lambda = 0.4$, the proposed algorithm achieves the optimal clustering result.

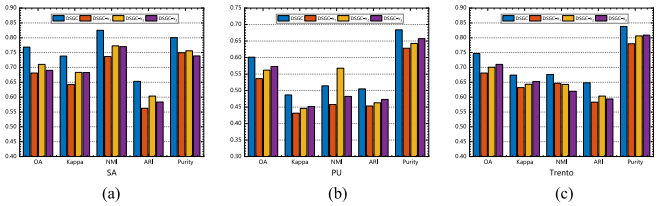


Fig. 9. Ablation experiment results on three datasets with five metrics. (a) on SA, (b) on PU, (c) on Trento.

E. Ablation Studies

This section assesses the role played by the local region generation, sparse graph construction, and dynamic GAE components in enhancing overall clustering effectiveness. Specifically, DSGC-V_1 removes the local region generation module (i.e., uses raw pixels instead of superpixels as graph nodes). The exclusion of dynamic GAE defines DSGC-V_2 (i.e., replaces it with a standard GAE with a fixed adjacency matrix), while DSGC-V_3 results from removing the sparse graph construction module (i.e., uses a fully connected graph). From the results shown in Fig. 9, we can observe that DSGC-$V1$ causes a notable drop in OA and ARI. This indicates that superpixel-based local region generation is crucial for preserving spatial–spectral coherence and reducing noise from isolated pixels, which directly benefits both OA and cluster assignment consistency. The effect is most pronounced on the PU dataset, where land cover is highly fragmented, suggesting that region generation helps to smooth over small-scale spectral variations. DSGC-$V2$ shows the most severe degradation across all metrics, particularly in NMI and kappa. This is expected, as the dynamic autoencoder is responsible for learning high-level semantic embeddings from the sparse graph structure. DSGC-$V3$ primarily affects purity and NMI on the Salinas and PU datasets. This is because sparse graph construction [see (16) and (17)] enforces a compact, discriminative connectivity pattern. Without it, the graph becomes dense or less structured, introducing noisy edges that blur cluster boundaries, thereby reducing the purity of detected clusters and the NMI. The effect is less severe on Trento, likely due to its simpler land cover distribution where even a dense graph retains reasonable cluster separation.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

TABLE V
RUNNING TIME AND MODEL COMPLEXITY OF ALL DEEP CLUSTERING METHODS ON THREE DATASETS

Name	Salinas			PU			Trento		
	Train/s	Test/s	FLOPs	Train/s	Test/s	FLOPs	Train/s	Test/s	FLOPs
EAGE	199.946	4.8399	270.38M	1763.686	39.8889	2.63G	186.1897	4.7581	57.09M
NCSC	681.0274	15.6768	303.18G	1375.568	33.2784	596.49G	677.5501	19.3583	273.67G
DAEGC	73.4302	5.1500	148.72 M	581.7761	44.8480	1.46 G	65.9399	4.7916	27.75 M
DFCN	672.1873	4.2803	490.69 M	1802.5410	8.5483	1.70 G	282.6037	4.4386	83.27 M
SDCN	437.5881	0.6087	446.1G	274.8834	0.2034	339.56G	145.6749	0.3492	236.22G
DSGC	126.1567	4.5571	281.09 M	1200.819	32.5107	2.19 G	95.7920	4.7351	61.26 M

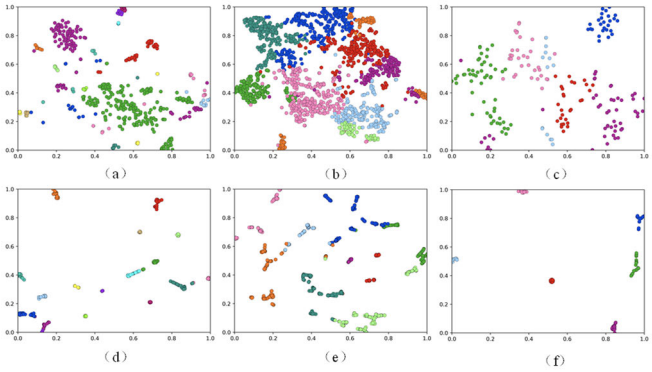


Fig. 10. t-SNE visualization of the DSGC on (a) and (d) SA, (b) and (e) UP, and (c) and (f) Trento. (a)–(c) Original distribution of three datasets. (d)–(f) Data distribution after clustering. Different colors in the maps denote different land covers.

F. Visualization Representations With t-SNE

To validate DSGC's clustering performance, the t-SNE technique [45] visualizes graph node distributions for three datasets within this section. Original distributions are depicted in Fig. 10(a)–(c), while Fig. 10(d)–(f) presents the distributions following DSGC processing. Key observations include: 1) lower interclass distribution covariance in Fig. 10(d)–(f) compared to Fig. 10(a)–(c), demonstrating effective clustering; 2) markedly greater distances between classes and diminished distances within classes in Fig. 10(d)–(f), reflecting a more compact and organized pattern; and 3) sharper differentiation among land cover types. These results confirm that DSGC produces enhanced intraclass compactness and interclass separation. Furthermore, the clustering task formulated in (7) is effectively realized.

G. Runtime and Model Complexity

Multiple deep clustering methodologies were examined to gauge model intricacy and operational efficacy, employing metrics such as FLOPs, training time, and inference delay. As indicated in Table V, DSGC demonstrates superior efficiency during both learning and evaluation stages relative to numerous comparable approaches. On the Salinas dataset, DSGC ranks second in OA (77.83%) while having the third-lowest training time (126.16 s) and the second-lowest FLOPs (281.09 M). DAEGC is faster (73.43 s) but yields substantially lower OA (70.66%). Conversely, NCSC achieves slightly higher OA (78.81%) but requires 5× more training time and 29 000× more FLOPs on limited platforms. DSGC's inference time is remarkably low (4.56 s on Salinas, 32.51 s on PU, and 4.74 s on Trento), thanks to the sparse graph structure that reduces neighborhood aggregation complexity to $O(n\alpha)$. This makes DSGC suitable for near-real-time inference after offline training, a desirable property for onboard processing where only forward passes are executed. The performance gap between DSGC and the best competitor varies by dataset. On Trento, DSGC outperforms the next best method (NCSC) by 3.45% in OA while requiring only 14% of NCSC's training time. On PU, where clustering is inherently difficult, DSGC achieves the highest OA (60.22%) with moderate cost (1200-s training and 2.19G FLOPs), still far lower than NCSC (1376 s and 84596G FLOPs). This indicates that DSGC's dynamic sparse updates are particularly beneficial for complex, fragmented scenes without incurring excessive overhead.

V. CONCLUSION

This article introduced DSGC, a novel DSGC Autoencoder framework designed for HSI clustering. The method began by employing local region generation for HSI preprocessing. This step retained crucial spatial–spectral structural details while simultaneously decreasing the graph's node count. Subsequently, an initial sparse graph was established, and a dedicated construction block modeled relationships between these regions. Then, a dynamic GAE extracted latent feature representations from this sparse structure to facilitate clustering. Iteratively during training, this autoencoder progressively enhanced graph sparsity through edge connection updates, optimizing pattern discovery. Finally, a joint optimization module enforced structural consistency, encouraged graph sparsity, and coordinated network training. This cohesive strategy ensured unified graph representations and systematically refined the architecture to boost clustering outcomes.

Future research will investigate additional spatial–spectral self-supervised techniques for graph learning. Furthermore, approaches such as reinforcement learning will be explored for HSI preprocessing to augment the feature extraction capabilities of clustering methodologies.

REFERENCES

[1] B. Xi et al., "Transductive few-shot learning with enhanced spectral–spatial embedding for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 34, pp. 854–868, 2025.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.

[2] C. Cheng, L. Zhang, H. Li, L. Dai, and W. Cui, "A deep stochastic adaptive Fourier decomposition network for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 33, pp. 1080–1094, 2024.

[3] Y. Zhang, W. Li, M. Zhang, J. Han, R. Tao, and S. Liang, "SpectralX: Parameter-efficient domain generalization for spectral remote sensing foundation models," 2025, *arXiv:2508.01731*.

[4] Y. Ding et al., "SLCGC: A lightweight self-supervised low-pass contrastive graph clustering network for hyperspectral images," *IEEE Trans. Multimedia*, vol. 27, pp. 8251–8262, 2025.

[5] Y. Zhang, W. Li, W. Sun, R. Tao, and Q. Du, "Single-source domain expansion network for cross-scene hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 32, pp. 1498–1512, 2023.

[6] Y. Ding et al., "AF2GNN: Graph convolution with adaptive filters and aggregator fusion for hyperspectral image classification," *Inf. Sci.*, vol. 602, pp. 201–219, Jul. 2022.

[7] Y. Ding et al., "Adaptive homophily clustering: Structure homophily graph learning with adaptive filter for hyperspectral image," *J. Article*, vol. 63, Jul. 2025, Art. no. 5510113.

[8] J. Feng et al., "S4DL: Shift-sensitive spatial–spectral disentangling learning for hyperspectral image unsupervised domain adaptation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 9, pp. 16894–16908, Sep. 2025.

[9] Y. Cai, Z. Zhang, X. Liu, Y. Ding, F. Li, and J. Tan, "Learning unified anchor graph for joint clustering of hyperspectral and LiDAR data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 4, pp. 6341–6354, Apr. 2025.

[10] Y. Ding et al., "Unsupervised self-correlated learning smoothy enhanced locality preserving graph convolution embedding clustering for hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5536716.

[11] Y. Zhang, W. Li, W. Jia, M. Zhang, R. Tao, and S. Liang, "Cross-domain hyperspectral image classification based on bi-directional domain adaptation," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 35, no. 12, pp. 1–14, Dec. 2025.

[12] T. Kanungo, D. M. Mount, N. S. Netanyahu, C. D. Piatko, R. Silverman, and A. Y. Wu, "An efficient k-means clustering algorithm: Analysis and implementation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 7, pp. 881–892, Jul. 2002.

[13] X. Yang, M. Zhu, B. Sun, Z. Wang, and F. Nie, "Fuzzy C-multiple-means clustering for hyperspectral image," *IEEE Geosci. Remote Sens. Lett.*, vol. 20, pp. 1–5, 2023.

[14] R. Krishnapuram and J. M. Keller, "The possibilistic C-means algorithm: Insights and recommendations," *IEEE Trans. Fuzzy Syst.*, vol. 4, no. 3, pp. 385–393, Aug. 1996.

[15] A. Ng, M. Jordan, and Y. Weiss, "On spectral clustering: Analysis and an algorithm," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 14, 2001, pp. 849–856.

[16] A. Rodriguez and A. Laio, "Clustering by fast search and find of density peaks," *Science*, vol. 344, no. 6191, pp. 1492–1496, Jun. 2014.

[17] Z. Chen et al., "MDA-HTD: Mask-driven dual autoencoders meet hyperspectral target detection," *Inf. Process. Manage.*, vol. 62, no. 4, Jul. 2025, Art. no. 104106.

[18] X. Zhao, J. Ma, L. Wang, Z. Zhang, Y. Ding, and X. Xiao, "A review of hyperspectral image classification based on graph neural networks," *Artif. Intell. Rev.*, vol. 58, no. 6, p. 172, Mar. 2025.

[19] H. Zhang, H. Zhai, L. Zhang, and P. Li, "Spectral–spatial sparse subspace clustering for hyperspectral remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 6, pp. 3672–3684, Jun. 2016.

[20] H. Zhai, H. Zhang, L. Zhang, P. Li, and A. Plaza, "A new sparse subspace clustering algorithm for hyperspectral remote sensing imagery," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 1, pp. 43–47, Jan. 2017.

[21] R. Wang, F. Nie, Z. Wang, F. He, and X. Li, "Scalable graph-based clustering with nonnegative relaxation for large hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 10, pp. 7352–7364, Oct. 2019.

[22] F. Gao et al., "Prototype-based information compensation network for multi-source remote sensing data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025.

[23] F. Gao, X. Jin, X. Zhou, J. Dong, and Q. Du, "MSFMamba: Multiscale feature fusion state space model for multisource remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025.

[24] J. Peng, L. Li, and Y. Y. Tang, "Maximum likelihood estimation-based joint sparse representation for the classification of hyperspectral remote sensing images," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 6, pp. 1790–1802, Jun. 2019.

[25] R. Guan et al., "Contrastive multiview subspace clustering of hyperspectral images based on graph convolutional networks," *IEEE Trans. Geosci. Remote Sens.*, vol. 62, 2024.

[26] Y. Zhao and X. Li, "Deep spectral clustering with regularized linear embedding for hyperspectral image clustering," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023.

[27] Y. Cai et al., "Superpixel contracted neighborhood contrastive subspace clustering network for hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5530113.

[28] Y. Zhang, X. Wang, X. Jiang, L. Zhang, and B. Du, "Elastic graph fusion subspace clustering for large hyperspectral image," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 35, no. 7, pp. 6300–6312, Jul. 2025.

[29] G. Wu, M. A. A. Al-Qaness, D. Al-Alimi, A. Dahou, M. Abd Elaziz, and A. A. Ewees, "Hyperspectral image classification using graph convolutional network: A comprehensive review," *Expert Syst. Appl.*, vol. 257, Dec. 2024, Art. no. 125106.

[30] Q. Wang, Y. Miao, M. Chen, and Y. Yuan, "Spatial–spectral clustering with anchor graph for hyperspectral image," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022.

[31] Y. Zhang, Y. Wang, X. Chen, X. Jiang, and Y. Zhou, "Spectral–spatial feature extraction with dual graph autoencoder for hyperspectral image clustering," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 32, no. 12, pp. 8500–8511, Dec. 2022.

[32] Y. Ding et al., "Self-supervised locality preserving low-pass graph convolutional embedding for large-scale hyperspectral image clustering," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022.

[33] B. Peng, Y. Yao, J. Lei, L. Fang, and Q. Huang, "Graph-based structural deep spectral–spatial clustering for hyperspectral image," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–12, 2023.

[34] J. Lin, F. Gao, L. Qi, J. Dong, Q. Du, and X. Gao, "Dynamic cross-modal feature interaction network for hyperspectral and LiDAR data classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025.

[35] J. Lin, F. Gao, X. Shi, J. Dong, and Q. Du, "SS-MAE: Spatial–spectral masked autoencoder for multisource remote sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, 2023.

[36] S. Zhang, H. Tong, J. Xu, and R. Maciejewski, "Graph convolutional networks: A comprehensive review," *Comput. Social Netw.*, vol. 6, no. 1, pp. 1–23, Dec. 2019.

[37] X. Li, H. Zhang, and R. Zhang, "Adaptive graph auto-encoder for general data clustering," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 12, pp. 9725–9732, Dec. 2022.

[38] S. M. Holland, "Principal components analysis (PCA)," Dept. Geol., Univ. Georgia, Athens, GA, USA, Tech. Rep., 2008, vol. 30602, p. 2501.

[39] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.

[40] Z. Zhang, Y. Cai, W. Gong, X. Liu, C. Zeng, and G. Yu, "MMAGL: Multiobjective multiview attributed graph learning for joint clustering of hyperspectral and LiDAR data," *IEEE Trans. Geosci. Remote Sens.*, vol. 63, 2025.

[41] C. Wang, S. Pan, R. Hu, G. Long, J. Jiang, and C. Zhang, "Attributed graph clustering: A deep attentional embedding approach," 2019, *arXiv:1906.06532*.

[42] H. Zhang, P. Li, R. Zhang, and X. Li, "Embedding graph auto-encoder for graph clustering," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 11, pp. 9352–9362, Nov. 2023.

[43] W. Tu et al., "Deep fusion clustering network," *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 11, pp. 9978–9987, May 2021.

[44] D. Bo, X. Wang, C. Shi, M. Zhu, E. Lu, and P. Cui, "Structural deep clustering network," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 11, pp. 9978–9987.

[45] L. Van der Maaten and G. Hinton, "Visualizing data using t-SNE," *J. Mach. Learn. Res.*, vol. 9, no. 11, pp. 2579–2605, 2008.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 07,2026 at 08:09:13 UTC from IEEE Xplore. Restrictions apply.